



HTML/CSS II

講義資料 - 前期

本日の内容

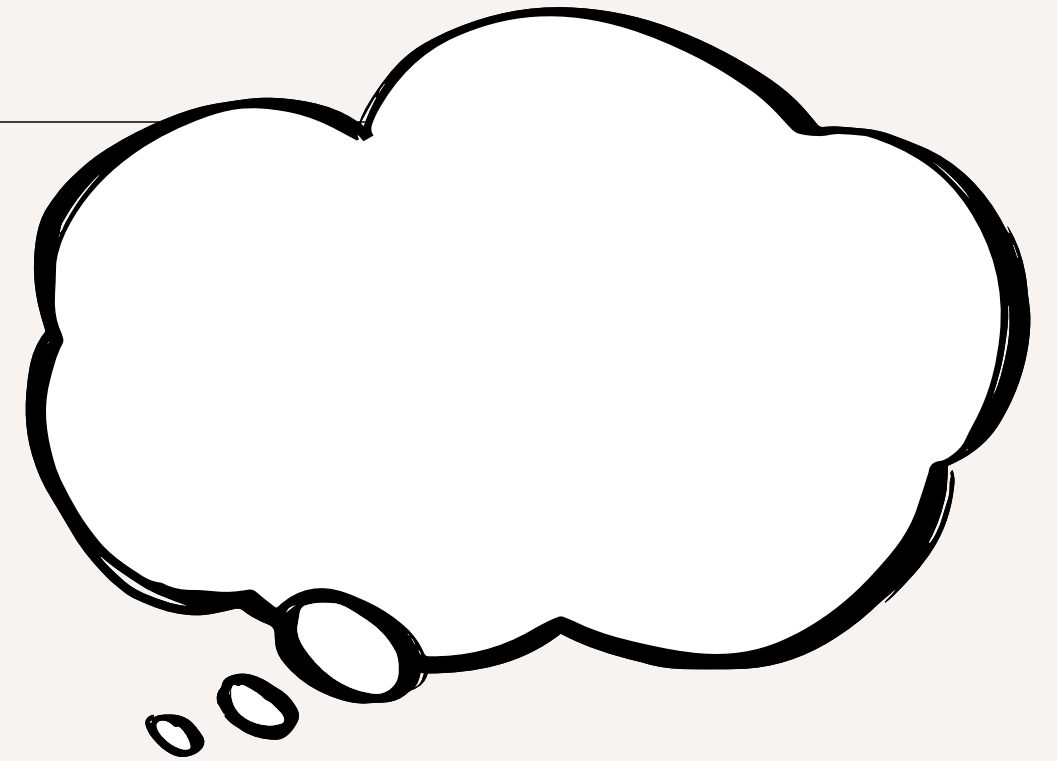
1. ウェブサイトとは何かを振り返ろう
2. Git系ツールをインストールしよう
3. DNSとは何かを振り返ろう
4. FTP・SFTPについて振り返ろう
5. サーバーの役割や設定を確認しよう
6. 開発環境を整えよう
7. まとめ

ウェブサイトとは？



良いデザインとは何か

改めて考えてみよう！



正解はないが...

デザインの目的

伝えるための「手段」であり、目的は別にある。

∴ユーザーの利益や体験を最大化しよう。

美しい映像がAmazonで流れたらいいデザインなのか。ゲームサイトなら？

デザインを具現化するのがコーディング作業

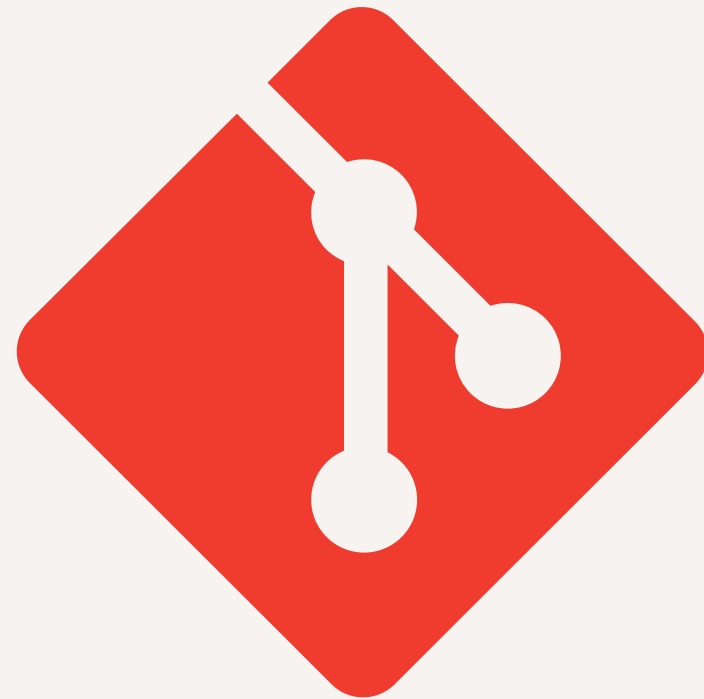
チラシなどとは違い、WEBで公開するためには「コード」で書く必要がある。

その方法としてHTMLを記述し、CSSで装飾(デザイン)を組み上げていく。

デザインとコーディングは双方向

使いやすさとは重さ(ファイルサイズや実行スピード)や色味、ユーザーの特徴など実感が伴うもの。デザイナーとコーダーは、原則コミュニケーションが必須となる。

Git系ツールを インストールしよう



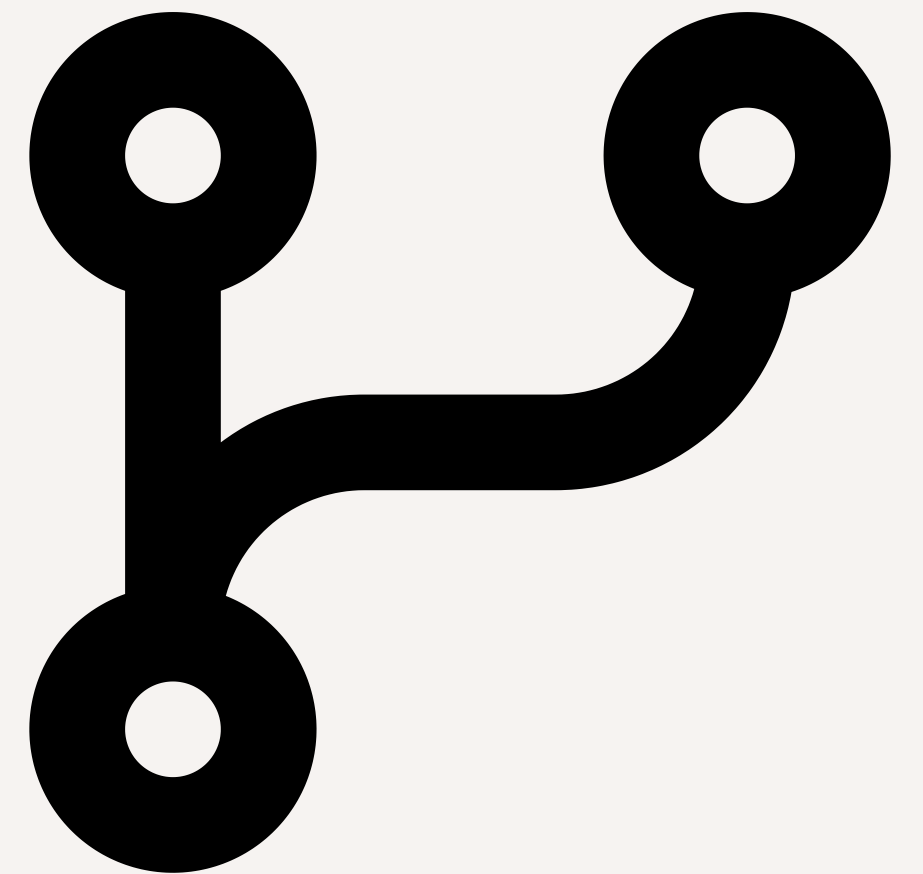
git

Gitとは？

分散型のバージョン管理システム。各コンピュータにデータを保持しながら、サービスと組み合わせることでクラウドで複数人から参照が可能。

【Gitの利点】

- 複数人で作業がしやすい
 - レビューを置けばダブルチェックも可能
- いつ、だれが、何をしたかが一目で分かる
- 過去のデータを保持しており、すぐに戻すことが可能



GitHubに登録しよう

無料で使えるクラウド型のGit管理サービス。Git自体は単体でも使えますが、クラウドで保存・参照させるためにはサービス契約が必要になる。その筆頭が「GitHub」で、Microsoft社が買収した。

<https://github.com/>

SourceTreeをインストールしよう

Gitをグラフィカル(GUI)に使うことのできるアプリケーション。
今回は特別な理由がない限り、SourceTreeに内蔵されたGitを使う。

※使う予定はないものの、インストール時に必要であればBitBucketアカウントを作成すること。

<https://www.sourcetreeapp.com/>

Macの初期設定

MacPCを使用している場合は、初回の認証でキーチェーンを求められる場合がある。
以下の手順に従って、「github」アカウントからクローンができるようにすること。

1. 「sourcetree」アプリメニュー→設定→アカウント→新規アカウントの追加
2. 認証を「HTTPS」に設定し、種類は「github」に設定
3. Macのキーチェーン情報が求められた場合は、PCのパスワードを入力



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

DNSとは何かを振り返ろう



DNSとは？

「ドメイン名」とインターネット上の住所にあたる「IPアドレス」を変換するための仕組み。文字の羅列では人間には難しいため、具体的な名前を付けることで認識しやすくなる。

入力情報	
対象ドメイン または IPアドレス	
google.com	
入力の逆引き または 正引き	
142.250.207.14	

CMAN 株式会社シーマン, 「ドメイン/IPアドレス サーチ
【whois情報検索】」,
(<https://www.cman.jp/network/support/ip.html>)

FTP・SFTPについて 振り返ろう



FTPとは？

ファイルを転送に使われる通信プロトコル。具体的には、ローカルにあるファイルをサーバーにアップロードする際によく使われる。平文(暗号化されていない)のため取り扱いに注意が必要。

❗FTP+SSLの暗号化機能を持った「FTPS」を使うようにすること。

SFTPとは？

FTPと同じくファイルを転送に使われる通信プロトコル。「SSH」という仕組みを利用したファイル転送で、FTPとは通信の方法が異なる。

- ✓ SFTPの方が安全性が高い。
- ✓ SFTPに対応しているのであれば、優先的に使うこと。
- ✓ ファイルに直接アップロードする方法は、減っていく可能性に考慮すること。



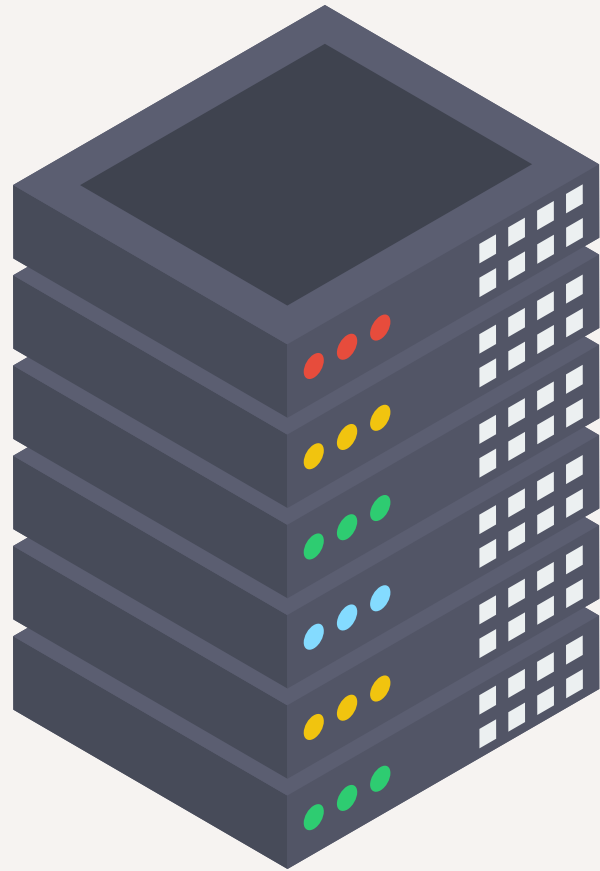
ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

小休止



サーバーの役割や 設定を確認しよう

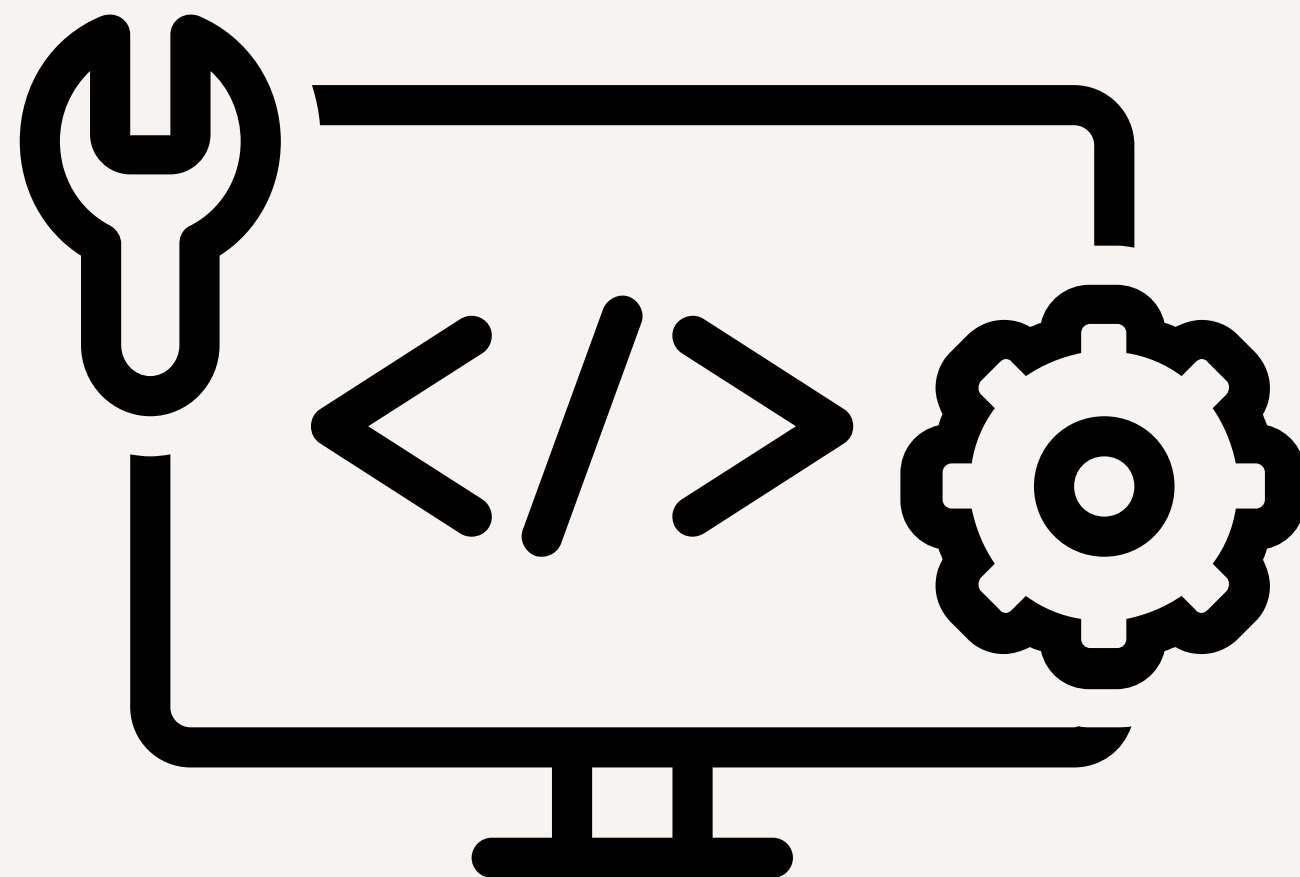


プランや契約状況を教えてください。

※クレジットカードなどの契約「情報」は不要。

- ドメインは有効か
- メールは使えるか
- DBは使えるか
- PHPのバージョン,他

開発環境を整えよう



「VSCode」の準備

無料エディタとして有名な「VS Code」を使う。プラグインなどは必要になった時に追加をする。今回は、以下のチェックリストを完遂する。

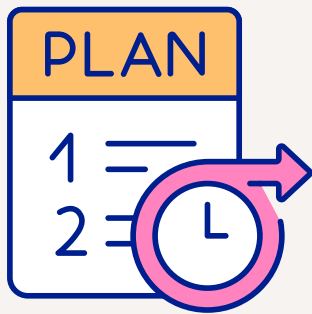
- ✓ ソフトウェアは最新版になっているか
- ✓ どのようなプラグインを入れているか
- ✓ ショートカットは使えるか(コピペなどの汎用やファイル検索など)
- ✓ 日本語化やモードの選択は適切か



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

今日の講義のまとめ



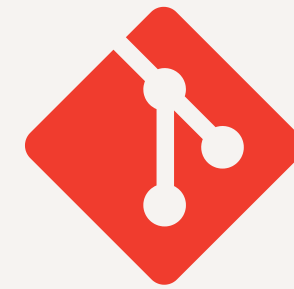
学習計画

大まかに今年度の取り組みについて紹介しました。今後も疑問点が出れば随時質問してください。



復習

WEB、DNS、FTPなど制作からサーバーに関する情報を復習しました。不明点は無くしておきましょう。



Gitの整備

これから使っていくツールとして「Git」を導入しました。必須級の強力なツールなので、主な機能は使えるようにしていきます。



開発環境の確認

エディタの使いやすさは、開発速度やバグの発生に影響します。自分に最適な環境を作っていきます。

WEBの知識確認など



- DartSassは何のために使ってますか(cssとの違い)
- コーディングルールなどは知っていますか
- jQueryについて説明してみてください
- 画像の形式について用途を説明してください
 - 写真、アイコン、ファビコンなど



HTML/CSS II

実習2 Gitの使い方／前年の復習
- HTMLマークアップとCSS構文の使い方

4月	5月	6月	7月
• 前期授業開始	※LP完了までJS止め	• LP制作完了(PF可)	
8月	9月	10月	11月
	• 後期授業開始		
12月	1月	2月	3月
	• WordPress制作完了 • 講義終了		

今回の内容

1. Gitの用語と使い方を学ぼう
2. Gitを実践してみよう
3. 今日の課題をgithubリポジトリから取得してみよう
4. プライベートブランチを作ってみよう
5. HTML／CSSの基礎問題を解いてみよう

Gitの用語と 使い方を学ぼう

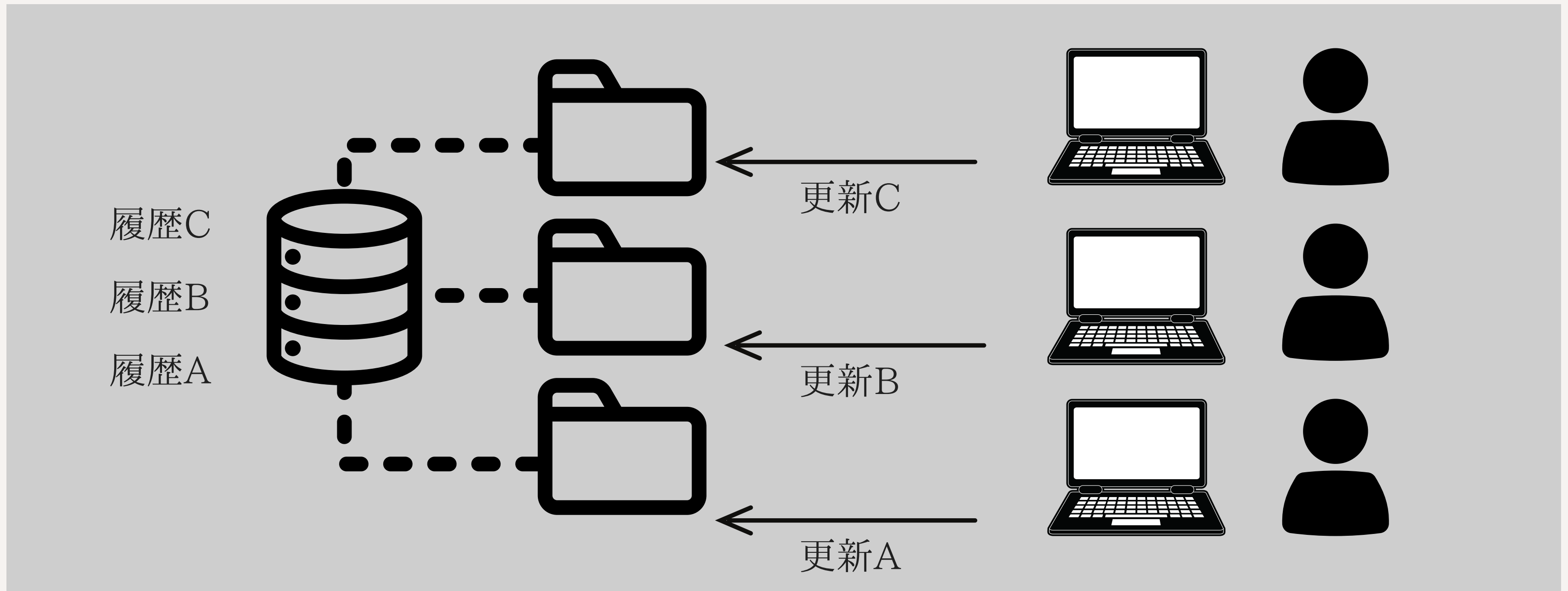


git

Gitの用語と使い方を学ぼう

分散型バージョン管理システム

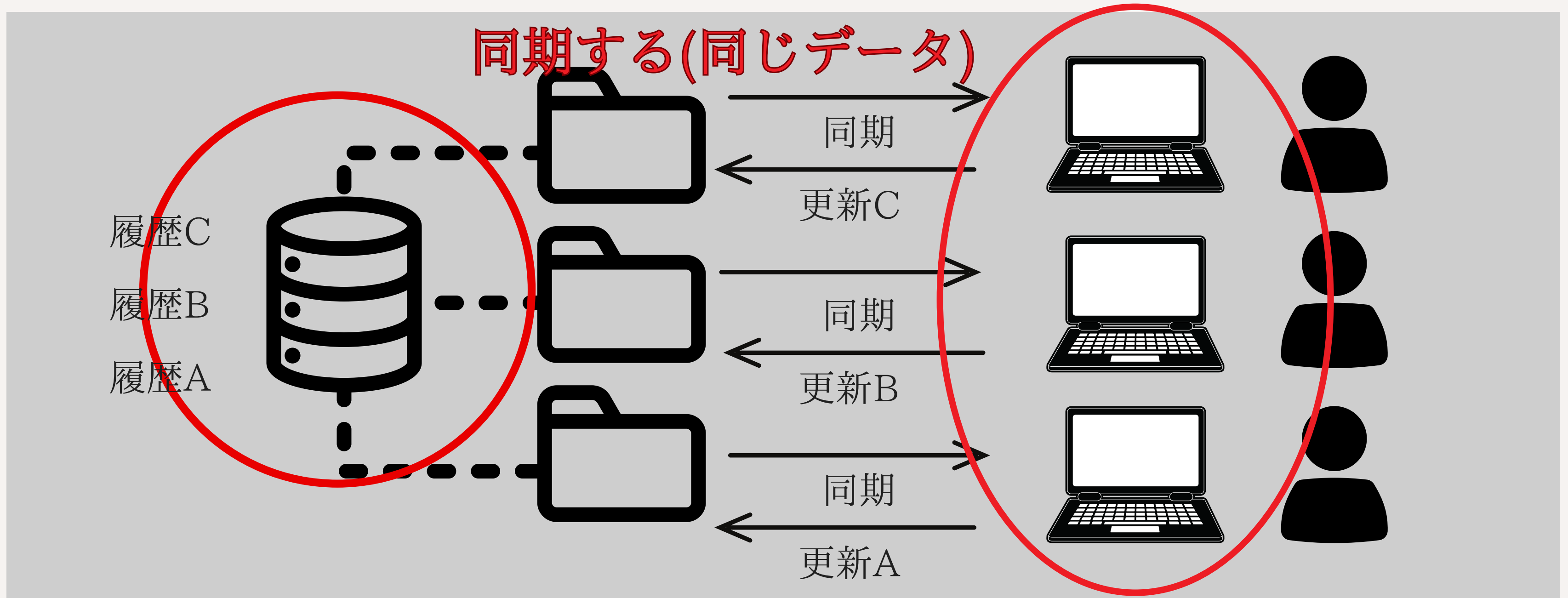
クラウドの共有ストレージと、各端末内の両方でデータを管理・共有するシステム。



Gitの用語と使い方を学ぼう

分散型バージョン管理システム

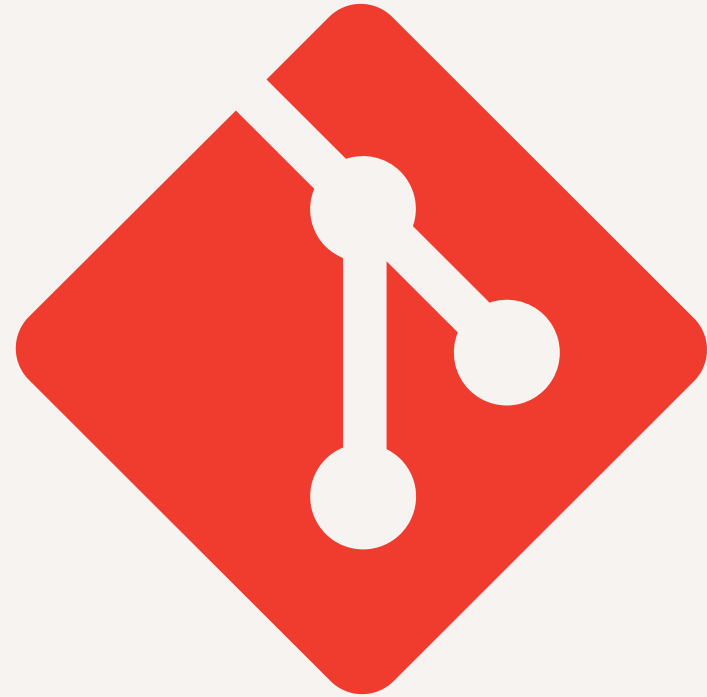
クラウドの共有ストレージと、各端末内の両方でデータを管理・共有するシステム。



主な用語集

- リポジトリ: ファイル自体や変更履歴、設定情報などが含まれたメインとなるデータベース
- 作業コピー／作業ツリー: リポジトリで管理しているファイルまたはツリー(機密など管理しないファイルも設定可能)
- コミット: 作業コピーの変更を履歴として保存する機能で、いわゆる1世代のデータにあたる
- チェックアウト: ローカルリポジトリのある時点でのファイルを取り出す、過去／別バージョンへの切り替えが可能
- ブランチ: 独立した作業ツリーで、分岐と合流を繰り返しファイルを更新する単位
- マージ: ファイルやブランチを取り込む(合流)し更新する作業
- コンフリクト: マージした際に同じファイルを更新しどちらが正か機械的に判断できなくなる現象、この場合は手動で修正をする必要がある
- クローン／フォーク: クラウドにあるリモートリポジトリをローカルリポジトリにダウンロード作成する作業
- フェッチ: リモートリポジトリとローカルリポジトリ間で更新があったかを確認する
- プル: リモートリポジトリの内容をローカルリポジトリに取り込む、フェッチ+マージを合成した機能
- プッシュ: ローカルリポジトリでコミットした履歴などをリモートリポジトリに反映させる作業
- プルリクエスト: ブランチ間のマージをクラウド上で発行する機能、これを通してコードレビューやドキュメント管理などとの連携がしやすくなる

Gitを実践してみよう



git



Gitを実践してみよう

テスト用リポジトリの作成

1. 「github」で「プライベートリポジトリ」として「test」を作成してみよう
 - a. パブリック設定にした場合は公開設定で誰でもアクセスできてしまいます(OSSなどはこの形式)
 - b. その他のオプションはデフォルトで良い
2. 「sourcetree」を立ち上げ、作成したリポジトリのデータをHTTPSで「クローン」する
 - a. 資格情報が求められたらgithubのユーザー情報を入力するかブラウザで認証する
3. ローカルに作成したリポジトリ直下に「test.txt」を作成し、「テストです」と入力して保存
4. 「sourcetree」上で変更を確認し、「ステージ」に追加した後メッセージを入力して「コミット」→「プッシュ」
5. 「github」に戻ると、「プッシュ」した内容が反映される
6. 「sourcetree」にもどり、「ブランチ」から「feature/first」ブランチを作成&チェックアウト
7. 「merge.txt」を新規で作成。「マージをテストします」など入力して保存し、「4」に倣い「プッシュ」まで実行
 - a. 「sourcetree」の☒マークを付けること(リモートに初めて登録する場合に使用する)
8. 「github」を開きグロナビの「Compare & Pull request」(以後PR)→内容を確認してPRを作成、そのまま
 - a. PRの元が「feature/first」、先が「main」で、「main」ブランチに合流するような操作になる
9. PRを承認し、「sourcetree」の「フェッチ」でリモート側が更新されたことを確認する(mainに「2↓」が表示)
10. 「main」ブランチをダブルクリックしチェックアウト、「プル」でリモートのデータをローカルに同期
 - a. 「プッシュ」と「プル」はローカル・リモート間で逆に近い動作をする

Gitを実践してみよう

コンフリクトを学ぶ

1. 「sourcetree」の「main」ブランチから新規で「ブランチ」「feature/conf1」を作成しチェックアウト
2. 先の項目で生成した「merge.txt」の2行目として「2行目のデータ」などを入力して保存
3. ステージに追加して「コミット」まで済ませる
4. 再度「**main**」**ブランチ**をチェックアウトし、新規で「ブランチ」「feature/conf2」を作成しチェックアウト
 - a. ※2行目の「データ1」が消えているいるので確認すること(=mainから分岐、枝分かれしたイメージ)
5. 先の項目で生成した「merge.txt」の2行目として「コンフリクトを確認」などを入力して保存
6. ステージに追加して「コミット」まで済ませる
7. 再再度「**main**」**ブランチ**をチェックアウトし、ここまでの一時停止
 - a. ※この時点の「merge.txt」は「マージをテストします」1行のみである
8. 今の「main」ブランチに、「feature/conf1」ブランチをマージ
 - a. サイドメニューの右クリック→「現在のブランチに〇〇ブランチをマージ」が分かりやすい、fast-fowardはOn
9. 続けて、「feature/conf2」ブランチをマージ
10. コンフリクトが起きるのでVSCodeで状況を確認してみる
11. 今回の場合は「Accept Both Changes」が適当で、両方の変更を取り込み内容の通りの行数になるよう調整
12. 保存が完了したら、「sourcetree
13. 「github」で画面を確認してみる **※このリポジトリは後で削除して構いません**

Gitを実践してみよう

その他のポイント

- 作業コピーはファイルの他、自動で設定される変更ブロック、行単位で変更できる
- PR上のコンフリクトはローカルマージでも解決可能(上位工程のため省略)
- コミットメッセージはルールがあると統一しやすい
 - 参考: <https://gist.github.com/joshbучea/6f47e86d2510bce28f8e7f42ae84c716>
- コミットの単位(タイミング)はプロジェクトごとに設定する
 - 一般には機能ごとと言われるが、機能の捉え方による
- バイナリファイル(テキストで判別できないExcelやAdobe系など)は差分が人間には判断できない
- 初心者時点での「リベース」や「リセット」機能は要注意、ただし良い機能ではある
- 設定によって管理しないファイルを設定できる

今日の課題をgithubリポジトリ
から取得してみよう





ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

今日の課題をgithubリポジトリから取得してみよう

ブランチの場所

- URL: <https://github.com/nichide-beni/html-css2>
- 上記URLからクローンまたはzipでダウンロードする
 - クローンの場合は今後の変更も追うことが可能
 - 書き換えのリスクがあるため、演習ファイルは各自分のディレクトリに管理すること
- クローンとは別で課題用のリポジトリとディレクトリを作ろう
 - リポジトリ: 「html-css-learn」で、「~/html-css-learn」に保存など(任意)

小休止



HTML／CSSの基礎問題を解いてみよう

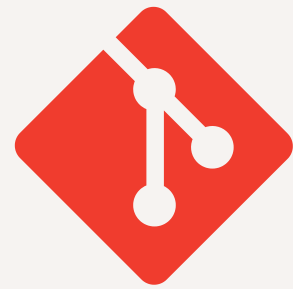


HTML／CSSの基礎問題を解いてみよう

設問項目について

-
- URL: https://docs.google.com/document/d/1AQxZZnv7ZgCb9fxHyT1qUUOF_-4ybNjIUIoc4nwoPD8/edit?usp=sharing
 - 権限のあるアカウントで閲覧した場合は、見ることが可能
 - 回答についてはリポジトリから取得したデータをコピーして使用すること

今回の講義のまとめ



Gitの使い方

基礎的な使い方とリポジトリのサービスを学び、これから使っていくための下地を整える。



復習と知識の確認

問題形式の課題を解くことによって知識の確認を行う。また未学習のコンテンツがあればこの場で身につける。

今回の内容

1. HTML／CSSの発展問題を解いてみよう

HTML／CSSの応用問題を解いてみよう

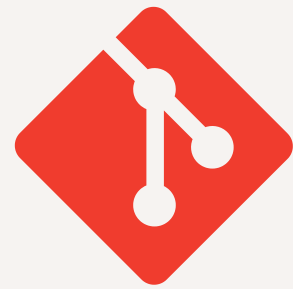


HTML／CSSの応用問題を解いてみよう

設問項目について

-
- URL: https://docs.google.com/document/d/1AQxZZnv7ZgCb9fxHyT1qUUOF_-4ybNjIUIoc4nwoPD8/edit?usp=sharing
 - 権限のあるアカウントで閲覧した場合は、見ることが可能
 - 回答についてはリポジトリから取得したデータをコピーして使用すること

今回の講義のまとめ



Gitの使い方

実際に自分のコンテンツとして使うことで、疑問点や習得したことを整理する。



他人のコード修正

一部コーディング済みのファイルを修正する作業を通し、他人のコードを修正する技術を習得する。



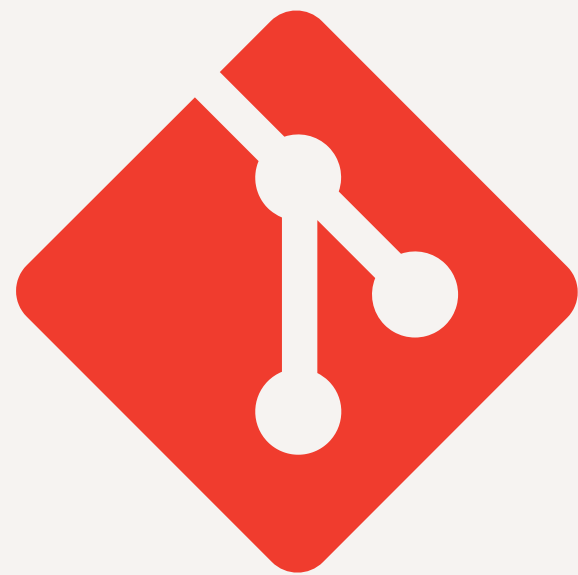
HTML/CSS II

実習3 Gitの使い方／前年の復習 - レスポンシブレイアウトのサイト制作

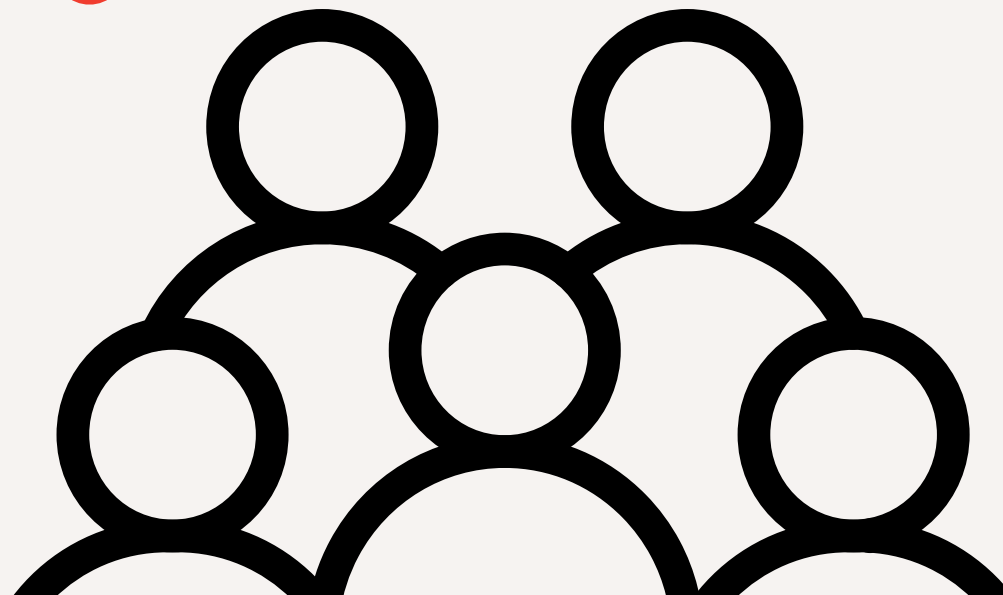
今回の内容

1. パブリックリポジトリを作ってみよう
2. レスポンシブウェブサイトの課題を作ってみよう

パブリックリポジトリ を作ってみよう



git



パブリックリポジトリを作ってみよう

回答用リポジトリの作成

1. 「github」で「プライベートリポジトリ」として「html-responsive-learn」を作成してみよう
 - a. パブリック設定にした場合は公開設定で誰でもアクセス可能
 - b. その他のオプションはデフォルトで良い
2. 「sourcetree」を立ち上げ、作成したリポジトリのデータをHTTPSで「クローン」する
 - a. 資格情報が求められたらgithubのユーザー情報を入力するかブラウザで認証する
3. 初期コミットなどは絶対ではありませんが、素材ファイル一式入れたらコミットしてみよう

レスポンスウェブサイトの 課題を作ってみよう



レスポンスウェブサイトの課題を作ってみよう

課題を確認して実践！

【課題】

課題は1つずつ行ってください。時間的には1つ終われば十分です。

- 課題1: <https://code-jump.com/store1-menu/>
 - 応用(課題1クリア後): 「Antique」の画像をカタログのようにタイトル上に持っていきましょう
- 課題2: <https://code-jump.com/corporate2-menu/>

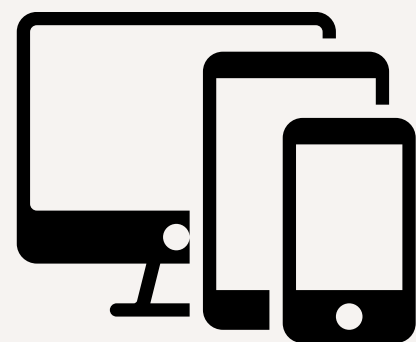
■ルール

- 1.sassを使っても使わなくても構いません
- 2.コーディングルールはフリーです
- 3.githubでの提出とします
- 4.他人に見られるソースという意識を持ってみてください(まずは少しで構いません)

■その他

- 本来はgithubで共同参加者として振る舞いますが、今回は割愛します
- 表示スピードは考慮しません

今回の講義のまとめ



マルチデバイス対応

主流のレスポンシブサイトを復習するとともに、自身のコーディングスタイルを認識する



HTML/CSS II

実習4 モダンなCSS技術を学ぼう

今回の内容

1. HTMLの便利な要素や書き方を知ろう
2. HTMLのテクニックを学ぼう
3. cssでできること・できないことを整理しよう
4. cssの便利なプロパティを学ぼう

HTMLの便利な要素や書き方 を知ろう

HTMLの便利な要素や書き方を知ろう

HTML Living Standardとは？

- HTML5.2以降のナンバリングではない最新のHTML仕様
- 仕様を策定する機関が変わっている
 - ただし、基本的にHTML5を引き継いでいるためほとんど同じように記述可能
- 反映スピードはナンバリング時代よりも高くなりやすい
 - →企業開発からOSSに変わったようなイメージ
- ちなみに、cssの仕様は旧HTML5を管理していた「W3C」が継続して仕様の策定



HTMLの便利な要素や書き方を知ろう

styleタグの取り扱い

`<style></style>`はどこに書けるのか？

HTML Living Standardではheadタグに書くように定められている

参考: <https://html.spec.whatwg.org/multipage/semantics.html#the-style-element>

HTML5.2ではbodyタグ(実質ほとんど)にも書ける仕様に改定されていた



【html-css2リポジトリ】「examples/chapter4/4-1/index.html」を開いてみよう



何色になるか答えてみよう

HTMLの便利な要素や書き方を知ろう

pictureで画像を出し分けよう

picture要素とは？

0以上の「source」要素と1つの「img」要素で構成されるタグ(sourceは上から順に読む)。
高解像度の画像や、レスポンス対応など幅広く対応できる。

使用例 - webp対応)

```
<picture>
  <source srcset="img/test.webp" type="image/webp" />
  
</picture>
```

使用例 - レスポンス対応)

```
<picture>
  <source srcset="img/test-sp.jpg" media="(max-width: 768px)" />
  
</picture>
```

使用例 - 高解像度の対応)

```
<picture>
  <source srcset="img/test.jpg, img/test@2x.jpg 2x" />
  
</picture>
```



【html-css2リポジトリ】「examples/chapter4/4-2/img/」を開いてみよう(htmlではなくディレクトリの方)



全部組み合わせたらどうなるだろうか？

※imgディレクトリを参考に「problem.html」を解く



開発者ツールのネットワークを開いてみよう



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

HTMLのテクニックを学ぼう

画像を遅延読み込みさせよう

遅延読み込みとは？

- ユーザーの表示速度改善や副産物としてのSEO対策などを目的とした技術の一つで、必要な時に必要なものを読み込ませるもの。
- 主に画像や動画など、容量の重いリソース系統に使用する。
- 「loading」属性で指定し、「lazy」が遅延読み込み、「eager」はデフォルトの即読み込み



【html-css2リポジトリ】「examples/chapter4/4-3/index.html」を開いてみよう



コードを読んで、どのような順番になるか予想してみよう



開発者ツールのネットワークを開いてみよう



- 「lazy」属性を使う場合は「width」と「height」を指定しないと表示に悪影響が出る場合がある(再レンダリング)
- 初期画面(FirstView)の画像には「lazy」を指定しないほうが表示が安定する(詳細はLCPで検索)
- しばらくは「src」属性よりも「loading」属性を先に書く(※Firefoxのバグが以下のサイトで表示されている間)

◦ <https://developer.mozilla.org/ja/docs/Web/API/HTMLImageElement/loading#%E4%BD%BF%E7%94%A8%E4%B8%8A%E3%81%AE%E6%B3%A8%E6%84%8F>



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

小休止



CSSでできること・できない ことを整理しよう



今回はネイティブcssを使います

CSSでできること・できないことを整理しよう

CSSは進化している

CSSの“常識”を見直そう

- 変数は**使用できる**
- ネスト(入れ子)による**プロパティ指定ができる**
 - &を使ったネストやメディアクエリも**指定可能**

■CSSはある程度ルールが少ないためリファレンスによって理解を深めやすい

<https://developer.mozilla.org/ja/docs/Web/CSS>

CSSでできること・できないことを整理しよう

ドキュメントを読んでもみよう

CSSでの変数利用について

sassと同じように、現在のCSSでは変数が使えるようになった。
書式にルールがあるので、読んで理解しよう。

https://developer.mozilla.org/ja/docs/Web/CSS/--*

※前で開いて説明します



【html-css2リポジトリ】「examples/chapter4/4-4/index.html」を開いてみよう



【html-css2リポジトリ】「examples/chapter4/4-4/media.html」を開いてみよう



結果について気づいたことを確認しよう

CSSでできること・できないことを整理しよう

ドキュメントを読んでもみよう

CSSでのネストについて

ネストについても、CSSではすでに有効化されている。

細部では違いがあるため、違いとともに使い方を理解しよう。

https://developer.mozilla.org/ja/docs/Web/CSS/CSS_nesting/Using_CSS_nesting

https://developer.mozilla.org/ja/docs/Web/CSS/CSS_nesting/Nesting_at-rules



【html-css2リポジトリ】「examples/chapter4/4-5/index.html」を開いてみよう



コードを読んで、コメントに従って回答を記述してみよう



ここまでで
質問はありますか？

遠慮せずに挙手・発言してください！

cssの便利なプロパティを学ぼう



今回はネイティブcssを使います

CSSの便利なプロパティを学ぼう

Gridレイアウト

横並びと位置揃え

display: gridでは、flex時代の横並びに加え縦位置の指定も容易になっている。
サンプルファイルを通して、理解を深めよう。



【html-css2リポジトリ】「examples/chapter4/4-6/index.html」を開いてみよう



単純な比率の横並びだけでなく、「subgrid」を使うことによって複雑な配置や旧来型ではJSを必要とするような記述も可能になった



覚えようとするとなかなか難しい可能性があるため、記憶に留めておいて引き出せるようにしておく。グリッドだけならライブラリも豊富なため、無理に作ろうとしないこともコツ。

CSSの便利なプロパティを学ぼう

画像レイアウト

親要素などに合わせた画像の大きさ表示

旧来のcssではbackground-imageやjsで処理していたKV(FV)画像のような表示をcssで実装することができる。

- object-fit: 親要素のサイズあるいは固定サイズに対して、表示サイズや拡大・縮小設定を適用できる。
 - <https://developer.mozilla.org/ja/docs/Web/CSS/object-fit>
- aspect-ratio: アスペクト比を固定した状態で表示できる。複数要素を並べるような表示で真価を発揮する。
 - <https://developer.mozilla.org/ja/docs/Web/CSS/aspect-ratio>



【html-css2リポジトリ】「examples/chapter4/4-7/index.html」を開いてみよう



Instagramのような画像をメインとした表示方法などがイメージしやすい。
グリッドレイアウトと併用するとデザインとコーディングの両立がしやすい。



画像などの場合「loading」属性と組み合わせると、比率を保持したまま適切なタイミングで処理を行わせることが可能(background-imageは遅延読み込みに未対応で、HTMLとしても解釈されないためSEOにも弱い)。

CSSの便利なプロパティを学ぼう

コンテンツのセンタリング

marginによる従来のセンタリングの拡張

marginおよびpaddingにおいて、水平や垂直に対するプロパティが追加されている。

- 水平: (margin|padding)-inline
- 垂直: (margin|padding)-block

子要素による親要素のサイズ設定

従来ではflexboxやinline-blockを使って親子にそれぞれ設定していたcssが親要素でまとめられるようになった。

これにより、子要素が入れ替わったり更新が入ったりしても改修がかけやすくなっている。

- 親要素に以下を設定する「width: object-fit」

子要素の上下中央寄せ

従来ではflexboxやpositionを使って親子にそれぞれ設定していたcssが親要素でまとめられるようになった。

これにより、大きさを維持するほか、子要素が入れ替わったり更新が入ったりしても改修がかけやすくなっている。

- 親要素に以下を設定する「place-content: center」



【html-css2リポジトリ】「[examples/chapter4/4-8/index.html](https://examples.chapter4/4-8/index.html)」を開いてみよう

CSSの便利なプロパティを学ぼう

切り抜きの指定

三角形やリボン型などを画像に頼らず指定する

旧来でも実装できたものの、より複雑な指定が可能になった。polygon指定が汎用的。

- clip-path: polygon({{ ポリゴンパス }})
 - <https://developer.mozilla.org/ja/docs/Web/CSS/clip-path>
- 自力で書くのは難しく **サイト利用を推奨**
 - <https://bennettfeely.com/clippy/>



【html-css2リポジトリ】「examples/chapter4/4-9/index.html」を開いてみよう

CSSの便利なプロパティを学ぼう

レスポンシブなプロパティ指定

フォントやパディングなどを流動的に指定する

メディアクエリを使わずにレスポンシブなサイズを設定できる

- {{プロパティ}}: clamp(最小値, 可変値, 最大値)
 - <https://developer.mozilla.org/ja/docs/Web/CSS/clamp>



【html-css2リポジトリ】「examples/chapter4/4-10/index.html」を開いてみよう



計算方法を考えてみよう



0にはできないので注意

CSSの便利なプロパティを学ぼう

その他の指定など

拡張された疑似要素を見てみよう

- :has
- :is
- :where
- その他

<https://developer.mozilla.org/ja/docs/Web/CSS/Pseudo-classes>



jsに頼らない記述が可能なように拡張されている

CSSの便利なプロパティを学ぼう

取り上げた内容の一覧表

HTML

- style: LS版ではhead内に記載するルール。ただし、bodyに書いても動作はするほかCMSなどの使用例もある。
- picture: 条件によって画像自体や解像度(retineなど)の異なる画像を出し分けることができるタグ。
- 画像の遅延読み込み: imgタグに「loading」属性で指定できる。ファーストビューには向かない。

CSS仕様

- ネスト、メディアクエリ、変数などが使用可能になっている
 - DartSassとは使い方やルールに差があるため使い方は異なる。css単体で見ればsass版のほうが高機能

便利なcssプロパティ

- Gridレイアウト - display: grid
- 画像レイアウト - object-fit, aspect-ratio
- コンテンツの水平センタリング: ブロック要素のmargin(inline/block指定), 子要素に合わせたwidthのobject-fit
- コンテンツの垂直センタリング: place-content: center
- コンテンツの切り抜き: clip-path(サイト利用推奨)
- min/max複合の割合指定: clamp
- 追加の疑似要素: :has,:is,:where

今回の講義のまとめ



便利なものはどんどん活用する

HTML/CSSは日々進歩しているため、リファレンスを時々見てみよう。他の言語に比べれば仕様は少なく、ブログ記事などを除いてみるだけでも進歩が期待できる。



HTML/CSS II

実習5 Gitの実習／前年の復習 - 制作したサイトの調整を試みよう(リファクタリング)

今回の内容

1. サーバーにファイルをアップしてみよう
2. リファクタリングとは？
3. LPテーマを決めてみよう

サーバーにファイルをアップしてみよう

FTP(S)を使ったアップロード

アップロードディレクトリ

- 公開ディレクトリ直下(ドキュメントルート)に置くと、最も使いやすい
 - マルチサイトを管理すると弊害が出るためその場合は考慮する
 - サブドメインを使うときれいにまとまるパターンも多い(今回は不採用予定)
- 個人情報などのファイルはアップしない
- sass等のソースファイルは任意、環境ファイルは要注意

FTPとFTPSの違い

- ポートやホスト情報は同じ
- SSLに対応するかの違いで、原則FTPSにすること

リファクタリングとは？

改修作業との違い

改修とは

- 既存の機能を変更すること
- 「修正」としてまとめて表記されることもある
- 広義の意味では機能追加としても利用される

リファクタリング

- 動作を変えずにコードの品質を高めること
- 神経質にならずに対応することが大事
- 古いコードや構造を変えるコードを無理に変える必要はない

例えば、重複や上書きから手を付けてみる、など

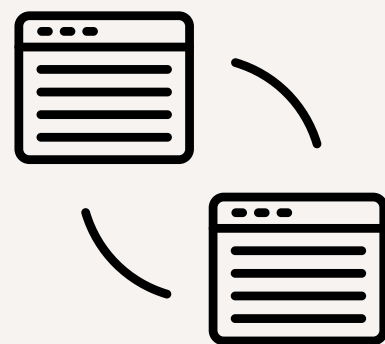
LPテーマを決めてみよう

目的を明確にしよう

- 自身のスキルを公開する案件は少ない
 - サービス型が汎用的であるが、フリーランスなら相手からのお問い合わせを受ける関係上公開する手もある
- 名刺や提出物にQRコードを貼っておくとアクセスがしやすい
- →教科書のタイプを見てみよう

参考: <https://be-marke.jp/articles/knowhow-making-persona>

今回の講義のまとめ



リファクタリング

必ずしも新規とは限らないため、既存の
ものを見直す技術は必要。本来的には、
既存のものにバグが出ないように見通し
を良くすると結果期間を圧縮できる。



HTML/CSS II

実習6 フレームワークを使ってみよう(Bootstrap)

今回の内容

1. Bootstrapとは？
2. フレームワーク(WEB)を使うメリット・デメリットについて
3. Bootstrapの(はじめに)を読んでみよう
4. ポートフォリオの参考事例と課題について

BootStrapとは？

WEBフレームワーク



※フレームワークとは

WEBにおいては、システムやサイトを作る際の枠組み/土台を指すもの。デザインなどの再利用だけでなく、システムにおいてはファイルの保存方法やセキュリティの仕組みなど広義の機能を持つ。

BootStrapとは？(5.3)

- HTML/CSS/JavaScriptを包括したフレームワーク
- 自分でCSSを書くことなくレスポンシブなグリッドデザインを実現できる
- モーダルをはじめとしたよく使われる動きについてはJSが用意されている
 - 簡単な動きならHTMLに属性を指定するだけで使用できる
- sassを使った場合グリッドのベースを変更したりテーマカラーを調整したりとカスタマイズがしやすい

フレームワーク(WEB)を使うメリット・デメリットについて

使うべき時と避けるべき時



メリット

- ベースを自分で組むことなく使うことができる
- 最新のバージョンを使えば一定のレベル、流行を担保できる
- 協力してサイトを構築する場合にルールの共通化がしやすい
 - スタートアップしやすい
- 初心者でも扱いやすい

デメリット

- 内部でルールが定まっている場合は利点が大きく削がれる
- デザインが偏りが出やすい、引っ張られる、類似する
- 最新のトレンドは追にくい
 - 短いサイクルで運用するものにはデメリットのほうが表に出る可能性がある
- 無駄なデータが多い

Bootstrapの(はじめに)を読んでみよう

はじめに

Bootstrap5.3

<https://getbootstrap.jp/docs/5.3/getting-started/introduction/>

- CDNを使った場合: 外部に通信を投げるため複数アクセスに強い
- 自サイトに組み込んだ場合: 外部サイトが落ちる、速度低下などが影響しない

デモサイトを動かしてみよう

<https://codepen.io/team/bootstrap/pen/qBamdLj>

Bootstrapのライブラリには何があるか？

CSSファイルとJSファイルで構成されており、用意されたクラスやJS用の属性をHTMLに書いて適用していく。CSSについては機能別に分割されている。JSはポップアップ(Popper)のみ分割。

<https://getbootstrap.jp/docs/5.3/getting-started/contents/>

Bootstrapの(はじめに)を読んでみよう

Bootstrap5.3をカスタマイズしよう

「html-css2」リポジトリ/examples/bootstrap5-site/

- base: ざっくりと型だけを記述したページ
- index: サンプルのカードサイト
- override: cssの上書き方法

BootStrapの(はじめに)を読んでみよう

BootStrap5.3をカスタマイズしよう

index.htmlをカスタマイズしてモーダル表示してみよう

「html-css2」リポジトリ/examples/bootstrap5-site/index.html



ポートフォリオの参考事例と課題について

自己紹介型のサイト

※「〇〇型」は便宜上の呼び方

- LIG(標準型)
 - <https://liginc.co.jp/629272>
- トライデントコンピューター(FirstView型)
 - <https://tridentwebdesign.blog.fc2.com/e/portfolio2024>
- 制作の参考
 - <https://note.himenoke.com/portfolio/>

【制作のポイント】

- 百聞は一見にしかずではないがPF<<<企業実績
- サイトを説明できるようになっておこう - ペルソナ設定など
- ターゲットに「刺さる」サイトを目指そう
 - どんな人を欲しがっているか
- 「合わない」作品は載せなくてもいい(専門家と要相談の上)

ポートフォリオの参考事例と課題について

課題 - 素材とテーマを決めよう

- 使う素材については集めておくこと
 - 自己作品がまとまってなければ集めておく、重いファイルは容量を軽くしておく
 - フリーアイコン、フリー写真などは適宜集める形で良い
- 参考サイトは決めておくか3つくらいに絞っておく

今回の講義のまとめ



フレームワークの使い方を学ぼう

長く親しまれるフレームワークライブラリ(Bootstrap)を通じ、活用方法と効率的なテクニックを学ぶ。Gridの登場によって以前ほどの優位性はない点は理解しておく。



ポートフォリオの概略を決めよう

ポートフォリオといっても様々な形がある。ズレがあると必ずしも加点にならない可能性もあることを考慮し、実績があれば優先的にアピールをする。



HTML/CSS II

実習7 フレームワークを活用したLPポートフォリオの制作実践(改変)

今回の内容

1. ペルソナを元にコンセプトをまとめよう
2. ワイヤーフレームを設計しよう

ペルソナを元にコンセプトをまとめよう

掲載する項目について

ページの順番を構成しよう

どのような順番で何を見せるかを検討する。

ペルソナを参考に優先度が高い方を先に置きたいが、文脈が崩れるなら順番は入れ替えること。

サイトとして独立させるが、別途提出する書類などと重複する内容は簡略することも検討する。

※複数ページの場合はサイトマップを先に作成し、TOPに流入を作る形式が簡単。

サンプル)

1.FV

2.制作に対する思い

3.実績・作品一覧

a.各作品のターゲット、ペルソナ、ユーザージャーニーなど背景情報があれば掲載しよう

b.企業実績などは明確な実績になるので積極的に採用しよう

c.自分の特長を活かしたオリジナル作品などは採用しよう(企業実績よりも優先しても良い)

4.保有スキル

その他、プロフィールなどを詳しく紹介する、個人の写真を掲載するなどフリーランスなどが採用するパターンなども存在する

ワイヤーフレームを設計しよう

ワイヤーフレームの設計

具体的なコンテンツの決定

ターゲットと掲載するコンテンツが決まれば、ワイヤーフレームを設計する。
既に制作済みの作品がベースになるため、素材をベースに組み立てを行う。



(一概には言えないが)一緒にやっていきたい、活躍できる長所をアピールしよう

内容のコンセプトを
決定

使用する素材(作品)
を選定

作品毎の背景を簡単
にまとめ再検討

作品数に応じてコン
テンツのバランスを
決定

全体のバランスを決
定



HTML/CSS II

実習8 Webやサーバーの知識を深めよう(改変)

今回の内容

1. DNSの理解を深めよう
2. Apacheの設定(.htaccess)でBASIC認証とリダイレクトを学ぼう
3. GTM/アナリティクスタグの使い方を学ぼう
4. キャッシュについて理解しよう
5. フォームの動作方法を学ぼう(概論のみ)

DNSの理解を深めよう

DNSとは？



DNSの役割

- Domain Name System の略で、ドメインをIPアドレスに変換する役割を持つ
 - Webサイトなどへのアクセスで、実際に使われるものがIPアドレス
 - プログラミング言語と機械語のような関係
 - 「example.com」 → 「100.100.100.100」 のように変換される
- DNSサーバーというものがあり、ドメイン名の問い合わせをされたらIPアドレスを返す仕組みがある
 - 種類があるが、レンタルサーバーレベルでは一部しか手を出せない領域にないと覚えておく

DNSサーバー

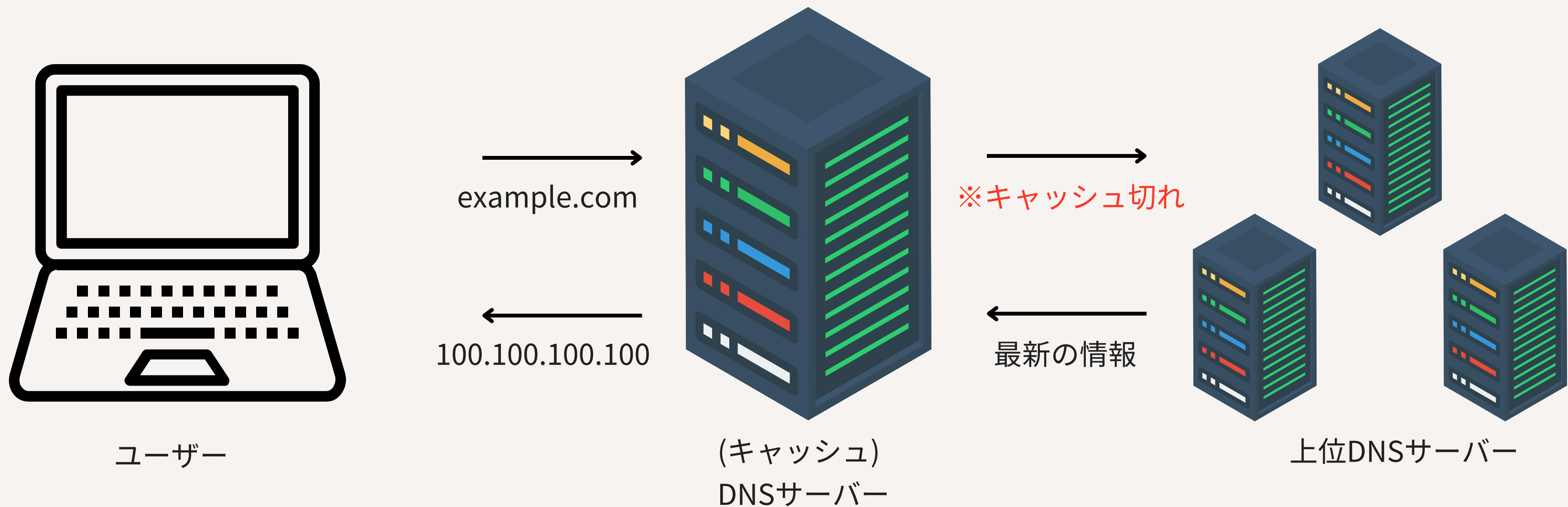
ドメインをIPアドレスに変換するためのサーバー。

ドメインの所有者が設定を変更できるが、実際に反映されるまでタイムラグが発生する(キャッシュ時間)。

✅ 古い言い方では「浸透時間」などといわれるが、現在は「DNSキャッシュ」「TTL」などのほうが適切

DNSの理解を深めよう

DNSのイメージ



DNSの理解を深めよう

DNSで管理するもの



DNSのレコード

- Aレコード
 - Webアクセスなど主軸となる役割を持つ。ドメイン名をIPv4アドレスに変換する。普段のネット検索で使っている。
- AAAAレコード
 - AレコードのIPv6版。IPv4枯渇問題への対応として実装。枯渇についてはまず大丈夫。
- MXレコード
 - メールサーバーの指定に使う。レンタルサーバーとして使う場合はデフォルトで設定されている場合が多い。サーバーと別のメールサービスを使いたい時に指定する(※1迷惑メール対策が別途必要)。
- CNAMEレコード
 - 別名を与える。具体的には「www.example.com」に「example.com」を設定しておくで、どちらでアクセスしても「example.com」の内容を返すことができる
- TXTレコード
 - その他のカスタム情報など入力でき、外部サービスの認証などに使われる。自身が所有者であることを表す場合によく利用される。
 - ※1: TXTレコードにSPF情報、DKIM、DMARCを設定すると迷惑メール率が減少する

DNSの理解を深めよう

DNSのよくあるトラブル



DNSを移管したい

- 有効期限がギリギリの場合は移管できない
- Whois公開代行などのサービス利用中は移管できない
- 登録アドレスに承認メールが届くため持ち主に注意

Webサーバーを変更したい

- 旧サーバーと新サーバーを見比べたい
 - constファイルなどを変更するか、CMSであれば表示用URLなど設定が必要
- サーバーが切り替わらない
 - TTLの時間が長いと更新されないことがあり可能であれば事前に短くしておく

メールサーバーを変更したい

- 切り替わり中のメールを取りこぼすときがある
 - ウェブメールが使える環境の場合は両方にログインで最も簡単
 - ウェブメールが使えない場合は、旧サーバーのIPとドメインの両方を受信すると実現しやすい

Apacheの設定(.htaccess)でBASIC認証とリダイレクトを学ぼう

BASIC認証とは？

※Apacheとhtaccessとは？

ウェブサーバーを構成するソフトウェアで、大きなシェアを誇る。htaccessはApacheの設定を上書きするためのファイル。ライバルのnginxでは使えないが、レンタルサーバーでは提供している場合がある。

✔ htaccessの方が設定が簡単で環境を提供しやすい。

✔ バージョンや設定によってそもそも上書きができない、書き方が違うなど利用するサーバーに左右される

BASIC認証

設定方法は違うが、Apache/nginx問わず利用できる。簡易的なパスワード認証。

入力のラグが非常に短く、**認証システムとしては脆弱**。あくまで簡易目的で利用すること。

例) リニューアル公開前のサーバーなど

Apacheの設定(.htaccess)でBASIC認証とリダイレクトを学ぼう

BASIC認証の設定方法

.htaccessを用意する

BASIC認証では、「.htaccess」と「.htpasswd」の2つを用意する。前者はApacheの汎用的な設定変更に使うため上書きなどには注意すること。

特に、記述を間違えるとサーバーエラーで繋がらなくなるためバックアップは必須。

```
AuthUserFile "/home/mmsxserv/mms-client.com/htpasswd/ncd-sc/.htpasswd"
```

```
AuthName "Member Site"
```

```
AuthType BASIC
```

```
require valid-user
```

- 絶対パス
- 人証明
- BASIC認証指定
- ユーザー指定(基本コレ)

.htpasswdを用意する

実際のIDとパスワードを記述する。

複数行に書くと複数のID/パスワードを設定できる。

平文では動作しないためパスワード設定サイトを利用すること。

<https://www.luft.co.jp/cgi/htpasswd.php>

Apacheの設定(.htaccess)でBASIC認証とリダイレクトを学ぼう

リダイレクトの設定

リダイレクトとは？

アクセスされた時に別のURLに転送すること。

- サーバーの引っ越し先に転送
- httpでアクセスされた時SSL対応のhttpsに転送
- wwwあり/なしを統一したい場合

SSLリダイレクトを実装してみる

RewriteEngine On

RewriteCond %{HTTPS} off

RewriteRule ^(.*)\$ https://%{HTTP_HOST}%{REQUEST_URI} [R=301,L]

※サーバーによって例外があるためAI活用などを推奨

- リダイレクト有効化
- 条件: HTTPSが無効の場合
- 全てのアクセスで: httpsのホスト/URI にリダイレクトして(R=301)それ以降の処理を終了(L)

GTM/アナリティクスタグの使い方を学ぼう

よく使うサイトツール(google)

Google Tag Manager(GTM)

タグを一括管理するためのツール。

単体で動作するものではないが、ソースコードを変更せずに管理画面での付け外し、バージョン管理が可能な必須級のツール。理由がなければ公開時に入れておくと楽。

Google Analytics

アクセスに関する情報を記録することができる最も有名なツール。

理由がなければ入れておくが無難。ただ入れるだけの場合は無用の長物にもなるため、今後の運用も決まっていて活用する余地がないなら不要。

Google Search Console

検索やサイトの最適化状態に特化した分析ツール。Analyticsのようにユーザーの行動を細かく見ることはできないが、何回表示されて何回クリックされたなどの結果が見える。タイトルの付け方、検索ワードなどの選定に使われる。ソースコード上に埋め込まなくてよく、パフォーマンスに影響がないため、これも入れておくが無難。予算を確保していないところでは使われていないところが難点。

キャッシュについて理解しよう

2つのキャッシュ

サーバーキャッシュ

開発者側が設定するキャッシュで、アクセス速度の向上や効率化などに活用される。
ユーザー側で采配できない代わりに、CDNなどの仕組みと組み合わせることで分散による負荷に強い構成が可能になる。



クライアントキャッシュ

各端末に保存されるキャッシュで、端末側からクリアすることで更新できる。
「更新したのに反映されません」の原因は大体こちら。
ブラウザ上の「スーパーリロード」または設定からの「キャッシュ」削除で反映できる。
※操作次第でログアウトがされることもあるので注意。

ブラウザによって差異があるが、Ctrl/Command + Shift + R によるクリアが汎用的。

フォームの動作方法を学ぼう(概論のみ)

フォームを扱う上の基本知識

HTML単体では入力値を保持できない

HTMLはユーザーへの情報を提供し、入力項目まで表すことができるが、入力値を保持することができない。

→ページ遷移は再接続になり、前の情報は原則クリアされる

- (PHPなどを使って)サーバーのセッションに値を保持する
- (PHPなどを使って)hiddenとして値を送信する(自分で書くのはおすすめしない)

JavaScriptではできないの？

P不可能ではないが最終的にはPHPなどのサーバーサイドの処理が必要。

処理も分割されてしまい優位性も特にないため、バニラのJavaScriptについては考えなくて良い。

ただし、バリデーションを使ってUI補助をする場合は度々ある。

フォームの動作方法を学ぼう(概論のみ)

フォーム入力とセキュリティ

サーバーサイド言語ではセキュリティ対策が必要

ユーザーの入力がサーバーに送られるため、生データでの処理は危険。

SQLインジェクションなどのセキュリティ問題が発生する。

→ユーザー入力がそのまま実行されてしまう。

サーバーサイドの処理をする場合は積極的にライブラリやフレームワークを採用し、バグの余地を残さないような体制づくりが重要になる。

